- Section 2.1 – Augmented Problem Statement

1. Description: This knowledge base system will simulate a journey inspired by the game, The Oregon Trail. The system models survival, resource management, and random events using the knowledge base and inference engine. The objective is to create an engaging and decision driven experience.
2. Example:
    - *Domain*: Interactive Rule Based Game Simulation
    - *Functional Requirements*: Must simulate a day by day progression, allow the player to make decisions, trigger rule based events, update player state, and determine win/loss conditions
    - *Non-functional*: Rule evaluation and response time should be near instant; The system must not crash during gameplay; Rules must be modular, readable, and easy to modify
    - *Constraints*: System is implemented entirely in CLIPS; Interaction is limited to text
    - *Edge Cases*: Multiple events occur during each other; simultaneous win/loss conditions

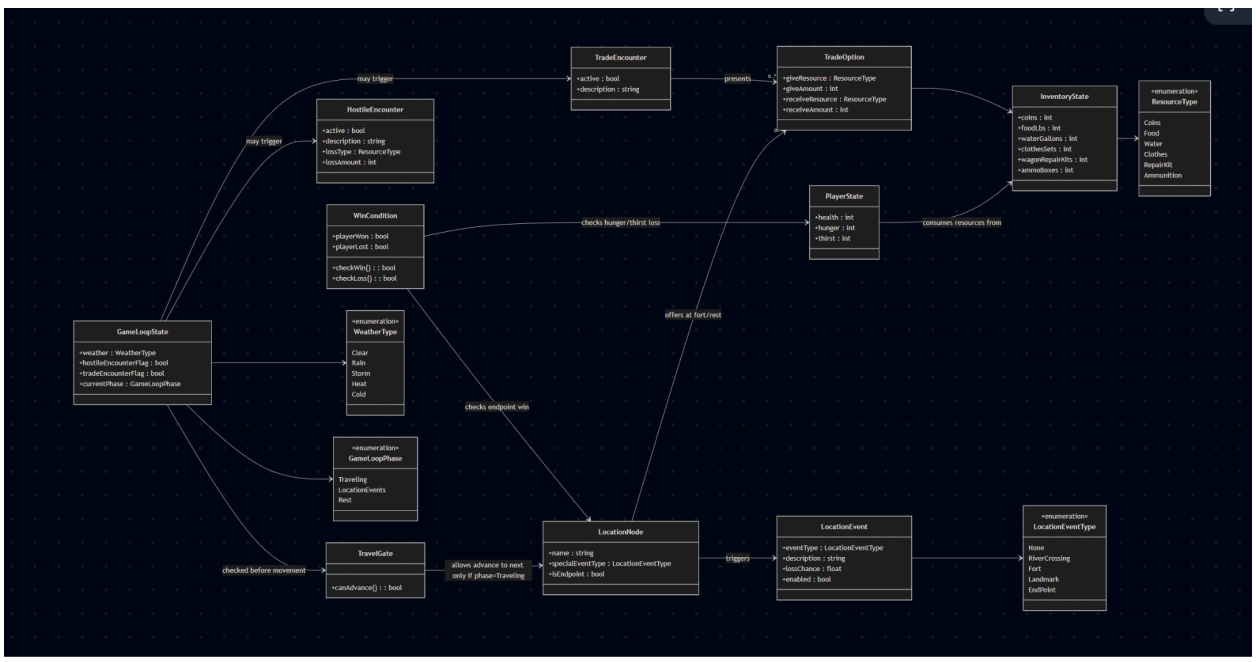
- Section 2.2 – Detailed Knowledge Representation & Ontology Scope

1. *Representation*: Entity-Oriented Approach with rules governing how they interact with other entities.
2. *Entities*:
    - Player State
        - Stores player health
        - Stores player hunger
        - Stores player thirst
    - Inventory State
        - Stores # of coins
        - Stores # of lbs of food
        - thStores # of gallons of water
        - Stores # of sets of clothes
        - Stores # of wagon repair kits
        - Stores # of boxes of ammunition
    - GameLoopState
        - Stores the days information:
            - Weather
            - Yes or No Hostile NPC Encounter
            - Yes or No Trade Encounter
        - Stores the current gameloop phase:
            - Traveling
            - =>
            - Location Events
            - =>
            - Rest
    - Location Node
        - Stores the current location
        - Stores special event information from each location. For ex there are river crossing with special events, or forts with special events.
        - Locations:
            - Independence, Missouri
            - Kansas River Crossing
            - Big Blue River
            - Fort Kearny
            - Chimney Rock

        - Fort Laramie
        - Independence Rock
        - South Pass
        - Fort Bridger
        - Green River
        - Soda Springs
        - Fort Hall
        - Snake River Crossing
        - Fort Boise
        - Blue Mountains
        - Fort Walla Walla
        - The Dalles
        - Willamette Valley (end point)
3. *Relationships*:
    - The GameLoopState will determine most of the interactions happening.
        - When it is traveling time each Location Node will be connected to the next location node so that the game can progress to the end point.
        - When the player reaches the end point in traveling the player wins.
        - When it is Location Events phase Location events for each Location Node will trigger. This includes river crossings where the player has a chance to lose a certain amount of equipment. Or fort events where the players can trade with the people in the fort.
        - When it is rest time the player will automatically eat food and drink water taking food and water from the inventory. When it is rest time the player also has the option to check for information about the current location, forage for food, and trade with locals.
        - If rest time ends and the player has not eaten or drunk the player will die causing the game to end.
    - Trading will be fired from specific triggers but mostly when the player decides to trade.

- Trading will consist of preset trade options where a random option will present itself to the player. Trade 3 pounds of food for 1 set of clothes or Trade 3 boxes of ammunition for a wagon repair kit.

4. *Constraints*:
    - Players can only trade in the resting phase or during a fort event.
    - Location Nodes must progress forward.
    - The GameLoopState must progress from traveling => location event => resting.
    - The player must travel to Willamette Valley to finish the game.
    - The player must be alive to finish the game.
5. *Diagram*:



https://imgur.com/a/pGpDtUt

- Section 2.3 – Inference and Reasoning Strategy

    1. Description: Details the reasoning mechanisms (forward/backward chaining, hybrid), conflict resolution, explanation generation, and performance optimizations.
    2. Example:
        - *Strategy*: Forward chaining for condition checks. Checks to see the current gamePhase, proceeds from there. There is not necessarily a solution so backwards chaining would not make sense.
        - *Conflict Resolution*: Using salience to have priority on certain events. Travel always takes priority, location events next, then any player actions, then rest/food consumption.
        - *Explanation*: The game will spill out information based on what's going on and inform the player of the current game state after each new location. Because it is a game this part is a bit more self-explanatory since you are just playing along with the game state.
        - *Optimization*: Quick conditioning events. Most events will be conditioned based on the current game state and the current location. Thus most rules will not trigger. Also the salience based approach ensures that one fact triggers at a time ensuring no explosion of events.
- Section 2.4 – Knowledge Acquisition and Validation Plan

    1. *Sources*:
        - Historical information about the Oregon Trail (migration routes, environmental conditions, etc)
        - Existing Oregon Trail-style games for gameplay mechanics and event design (Example)
        - Course materials on CLIPS, rule-based expert systems, and knowledge representation
        - Online documentation for CLIPS syntax
    2. *Elicitation*:
        - Break down gameplay into core mechanics:
            - Survival (food, water, clothing)

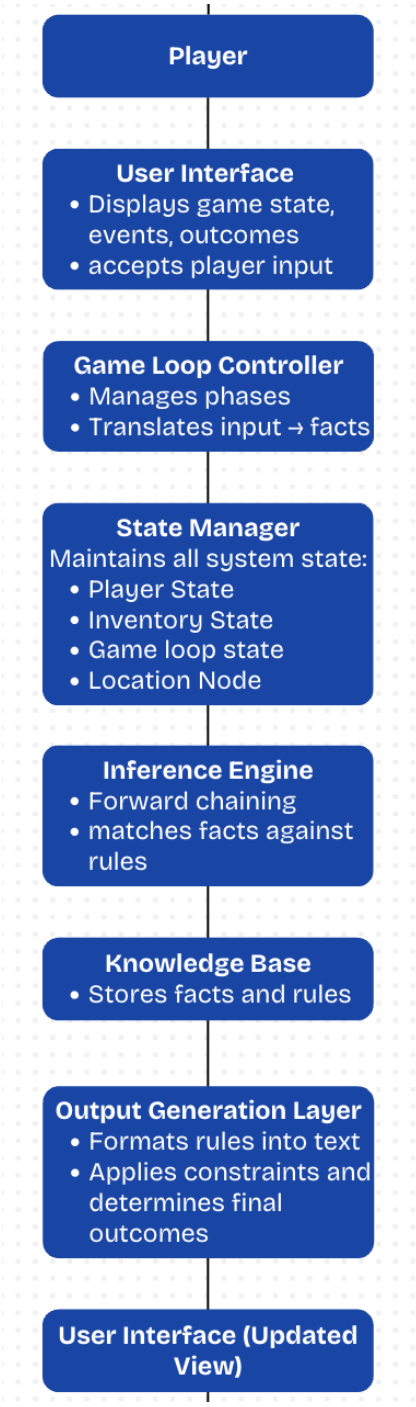
        - Travel progression (distance, locations)
        - Random events (weather, NPC encounters
    - Translate each mechanic into formal rules and facts with CLIPS
    - Group rules into categories:
        - Survival Rules
        - Environment Rules
        - NPC Interaction Rules
        - Game Progression Rules
    - Refine rules with incremental testing:
        - Begin with isolated rules
        - Then integrate into full game loop
3. *Validation*:
    - Unit Testing
        - Test individual rules in isolation (e.g. starvation rule triggers when food = 0, and travel progression updates distance correctly)
    - Scenario-Based Testing
        - Predefined gameplay scenarios (e.g. player survives with sufficient resources, player dies after multiple days without food, etc)
    - Full Simulation Testing
        - Run complete playthroughs to ensure that win/loss conditions trigger as expected, no rule conflicts or infinite loops occur, and the game loop executes correctly.
4. *Governance*:
    - Maintain versions of the knowledge base:
        - Ex: kv1_survival_rules, v2_npc_events
    - Track rule changes and updates
    - Perform weekly reviews to remove redundant rules and improve rule clarity
    - Final validation pass before submission to ensure consistency across all rules

- Section 2.5 — System Architecture & Integration

1. *Components*:

- Knowledge Base (ontology + rules)
    - Stores all facts and rules, like player state, environment state, game rules, etc.
- Inference Engine (CLIPS)
    - Executes forward-chaining reasoning
    - Evaluates rules based on current facts
    - Updates game state dynamically
- Game Loop Controller
    - Controls progression of the game, including start of day, player action phase, end of day updates, etc
- User Interface (UI)
    - Text-based command-line interface
    - Accepts player input (travel, rest, trade)
    - Displays game state updates, event outcomes, win/loss messages, etc

2. *Integrations*:
    - No external systems required
    - Runs entirely locally using CLIPS
    - No database, API, or dependencies
3. *Diagram*:



- Section 2.6 – Project Schedule & Milestones

    1. Description: Outlines timeline, major deliverables, and checkpoints for development, testing, and deployment.
    2. Example:
        - *Milestones*:

- Week 10: Planning finalized; preliminary development plan and design completed; knowledge base structure defined
- Week 11: Basic facts and rules implemented; inference behavior tested and verified
- Week 12: Game loop implemented; player choices integrated; state updates functioning
- Week 13: Event rules added; logic refined; win and loss conditions implemented
- Week 14: Randomness introduced; multiple gameplay scenarios tested
- Week 15: Full system integrated; complete playthrough testing conducted; rule consistency verified
- Week 16: Final testing and debugging completed; presentation prepared